

CockroachDB Architecture Guide

CockroachDB University | Version 26.1

Distributed SQL Architecture

CockroachDB is a distributed SQL database designed to scale horizontally while maintaining strong consistency and high availability. Unlike traditional databases that rely on a single primary node, CockroachDB distributes data across all nodes in the cluster symmetrically. Every node can accept reads and writes for any data, which eliminates single points of failure and enables linear scalability.

Data is organized into a sorted key-value map that is divided into contiguous ranges of approximately 512 MiB each. Each range is an independent unit of replication and distribution. As data grows, ranges automatically split; as data shrinks, they merge. This approach allows the system to balance load across nodes without manual intervention.

Replication and Consensus

Every range in CockroachDB is replicated to at least three nodes using the Raft consensus protocol. Raft ensures that a majority of replicas (a quorum) must agree on every write before it is considered committed. This guarantees that committed data survives any single node failure.

One replica in each Raft group is elected the leaseholder. The leaseholder serves all reads and coordinates all writes for that range. If the leaseholder fails, Raft automatically elects a new leader and transfers the lease, typically within a few seconds. Clients experience a brief increase in latency but no data loss.

The replication factor is configurable at the zone level. Critical tables can use a replication factor of 5 for additional durability, while less important data might use 3. Multi-region deployments pin replicas to specific regions to control data placement and access latency.

Transaction Model

CockroachDB implements serializable isolation, the strongest isolation level defined by the SQL standard. Serializable isolation prevents all concurrency anomalies: dirty reads, non-repeatable reads, phantom reads, and write skew. This eliminates an entire class of bugs that plague applications running on databases with weaker isolation.

Transactions use a multi-version concurrency control (MVCC) scheme with hybrid logical clocks (HLC). Each transaction is assigned a timestamp, and reads see a consistent snapshot of the database as of that timestamp. Write-write conflicts are detected eagerly and resolved through a combination of locking and transaction restarts.

When two transactions conflict, CockroachDB automatically retries the transaction that arrived later. Applications should implement retry loops to handle SQLSTATE 40001 errors. Client libraries for Go, Java, Python, and other languages provide built-in retry helpers.

Storage Engine

CockroachDB uses Pebble as its storage engine, a high-performance key-value store inspired by LevelDB and RocksDB. Pebble provides sorted key-value storage with LSM-tree-based compaction, write-ahead logging for durability, and efficient range scans.

Each CockroachDB node runs a single instance of Pebble. All ranges hosted on that node share the same Pebble store. Pebble handles background compaction to merge sorted run files and reclaim space from deleted or overwritten keys. The compaction strategy balances write amplification against read amplification.

MVCC garbage collection runs periodically to remove old versions of keys that are no longer needed. The GC TTL (default 4 hours) determines how long historical versions are retained. This retention window enables AS OF SYSTEM TIME queries and backup operations.